CSE 546 Reinforcement Learning

Spring 2025

Instructor - Alina Vereshchaka

Assignment 1: Defining and Solving RL Environments

Name: Rajasrivatsan Srinivasan

UB Person Number: 50604233

UB Mail: rajasriv@buffalo.edu

Introduction:

Reinforcement Learning (RL) is a machine learning approach where an agent improves its behavior through interactions within an environment. This assignment involves designing and implementing custom RL environments using the Gymnasium framework. The objective is to understand the core principles of Markov Decision Processes (MDP) by defining and solving RL environments.

For this assignment, I have selected and implemented **Warehouse Robot Environment**. This environment represents warehouse robots used for transporting goods while efficiently navigating obstacles. The design focuses on optimizing movement strategies to ensure smooth and effective operations.

Environment Overview

- The robot operates on a **6×6 grid**.
- Pick up an item from a **pickup location** and deliver it to a **drop-off location**.
- The robot must **navigate efficiently** while avoiding **static obstacles (shelves).**
- The environment rewards efficiency and penalizes unnecessary steps or collisions.

Part 1 : Defining RL Environments:

1. Deterministic Environment:

Set of Actions:

- **Up (Action 0)**: Move the robot up one grid cell.
- **Down (Action 1)**: Move the robot down one grid cell.
- **Left (Action 2)**: Move the robot left one grid cell.
- **Right (Action 3)**: Move the robot right one grid cell.
- **Pickup (Action 4)**: If the robot is at the pickup location and is not carrying an item, it picks up the item.
- **Dropoff (Action 5)**: If the robot is at the dropoff location and is carrying the item, it drops off the item.

Set of States:

- **Position**: The robot's (x, y) position on the grid.
- **Carrying**: A boolean that indicates whether the robot is carrying the item (True if it is carrying the item, False otherwise).
- Thus, the state space consists of a combination of these two aspects: the robot's position on the grid and whether it is carrying the item. So the total number of unique states is **36×2=72.**

Set of Rewards:

- **+100 points** for successfully delivering the item to the dropoff point.

- **+25 points** for picking up the item from the pickup point.
- **-1 point** for each step to encourage efficiency (penalty for moving).
- **-20 points** for hitting an obstacle (to discourage collisions with obstacles in the grid).
- **-5 points** for attempting to pick up or drop off an item at an invalid location (e.g., trying to pick up the item when not at the pickup point).

Stochastic Environment:

Set of Actions:

- Up (Action 0)
- Down (Action 1)
- Left (Action 2)
- Right (Action 3)
- Pickup (Action 4)
- Dropoff (Action 5)

Set of States:

- **Position**: The robot's position on the 6x6 grid.
- **Carrying**: Whether the robot is carrying the item (True or False).
- In the stochastic environment, the robot's state can still be represented as (position, carrying), but **random events** may alter the outcomes of its actions, introducing some uncertainty.

Set of Rewards:

- **+100 points** for successful delivery.
- **+25 points** for picking up the item.
- **-1 point** for each step.
- **-20 points** for hitting an obstacle.
- **-5 points** for invalid actions (e.g., picking up when not at the pickup point)

2.

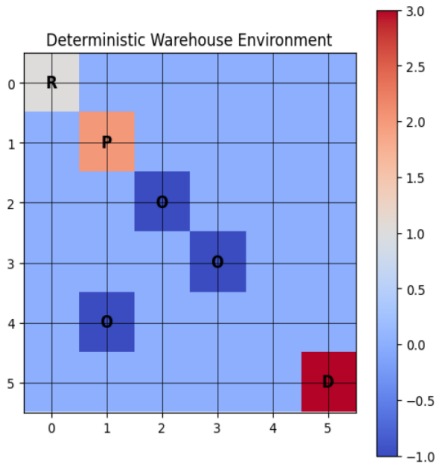


Fig 2.1 State: (0, 0, 0), Action: 0, Reward: -1

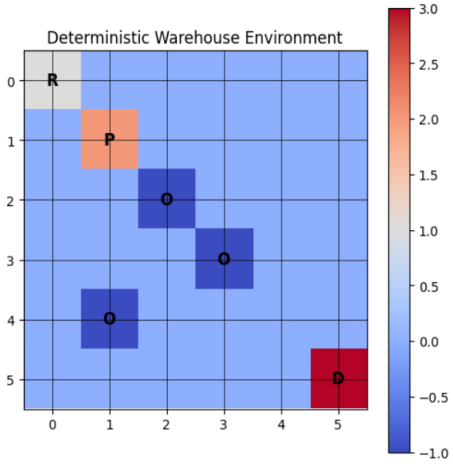


Fig 2.2 State: (0, 0, 0), Action: 2, Reward: -1

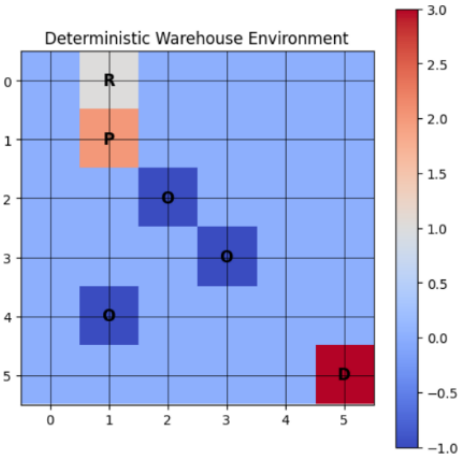


Fig 2.3 State: (0, 1, 0), Action: 1, Reward: -1

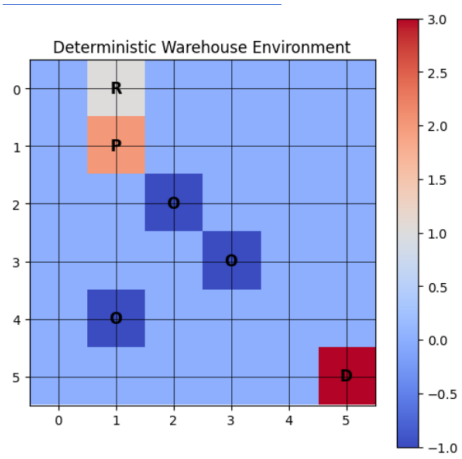


Fig 2.4 State: (0, 1, 0), Action: 2, Reward: -1

Stochastic Warehouse Environment:

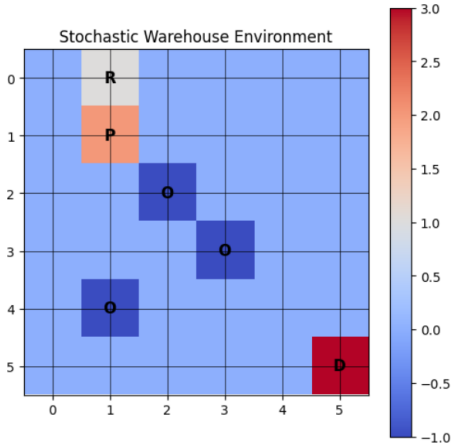


Fig 2.5 State: (0, 1, 0), Action: 1, Reward: -1

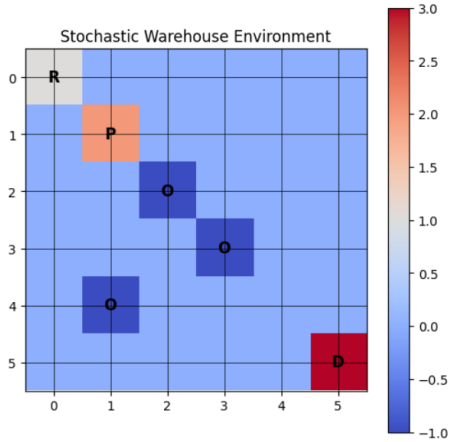


Fig 2.6 State: (0, 0, 0), Action: 0, Reward: -1

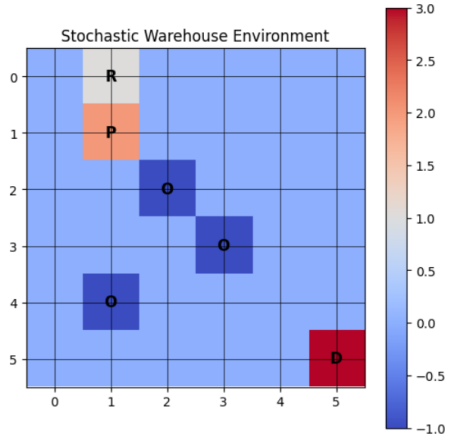


Fig 2.7 State: (0, 1, 0), Action: 1, Reward: -1

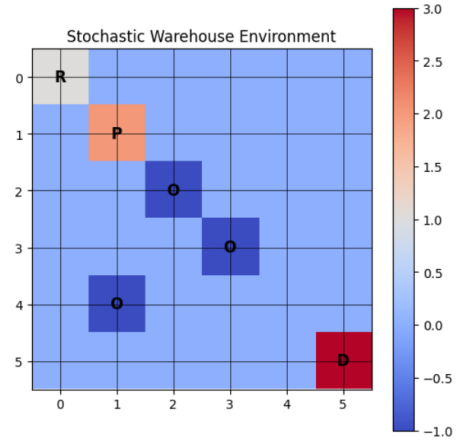


Fig 2.8 State: (0, 0, 0), Action: 0, Reward: -1

3. The stochastic environment was **defined by introducing random events** during the agent's movement. Specifically, there is a **10% chance that the agent's action** (moving, picking up, or dropping off an item) will fail, causing it to remain in the same position or not complete the action. This randomness simulates real-world uncertainties, like malfunction or external interference, **making the environment unpredictable** compared to the deterministic one.

4. In a deterministic environment, the outcome of each action is predictable and always the same when the agent takes the same action from the same state. There are no random elements affecting the result.

In a stochastic environment, the outcome of an action can vary due to randomness or uncertainty. Even if the agent takes the same action from the same state, the result may not always be the same, introducing variability and unpredictability.

Aspect	Deterministic Environment	Stochastic Environment
Predictability of Actions	Actions lead to the same result every time.	Actions may lead to different results due to randomness.
Rewards	Rewards are fixed and predictable.	Rewards vary due to random events (e.g., obstacles or external factors).
State Transitions	State transitions are consistent and predictable.	State transitions can change due to chance.
Environmental Behavior	Environment behaves consistently throughout the process.	Environment introduces uncertainty, affecting the agent's behavior.

5. To ensure the safety of the environment in my implementation, I have defined strict boundaries for the agent's actions. For example, the agent can only choose from predefined actions, such as moving up, down, left, right, picking up, and dropping off items, ensuring it doesn't take illegal actions. The environment also checks for obstacles, preventing the agent from moving through blocked areas.

Part 2: Applying Tabular Methods

1(a). Base Model: Running Q-learning on Deterministic Environment with default hyperparameter

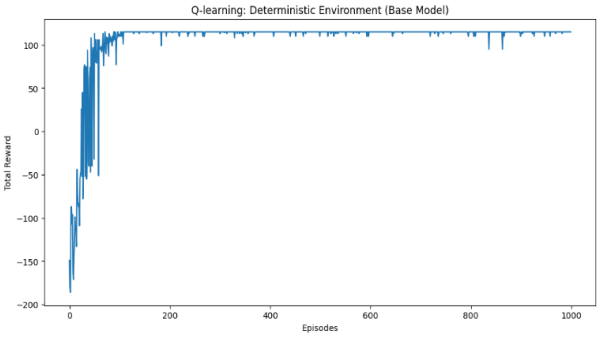


Fig 1.1 Q-Learning on Deterministic Warehouse

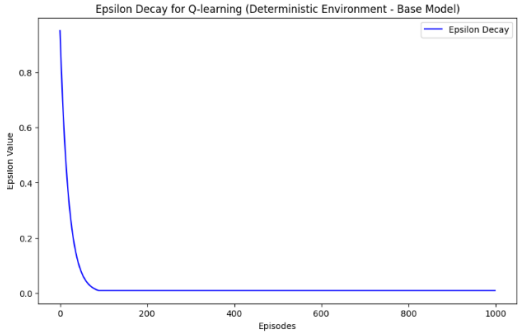


Fig 1.2 Epsilon Decay for Q-Learning

Observation:

Total Reward per Episode (Q-learning on Deterministic Environment):

- The graph shows the **total reward per episode** for the Q-learning algorithm. Initially, the total reward fluctuates, with several negative values observed in the early episodes. As the **episodes increase**, the **reward stabilizes**, reaching a higher positive value, indicating that the **agent has learned an optimal or near-optimal policy**. The sharp increase in total reward towards the end suggests that the agent has successfully exposed to the environment and learned to maximize its reward by choosing the most efficient actions.

Epsilon Decay (Q-learning on Deterministic Environment):

- The graph illustrates the epsilon decay over the episodes. **Epsilon starts at 1.0**, **representing maximum exploration**, and **decreases sharply** as the number of episodes increases. This is typical behavior in Q-learning, where the agent initially explores the environment and gradually shifts towards exploiting its learned knowledge.The epsilon value decays to a **minimum value (0.01**), reflecting the agent's reduced exploration as it becomes **more confident in its decisions**

1(b). Applying Q-Learning on Stochastic Warehouse

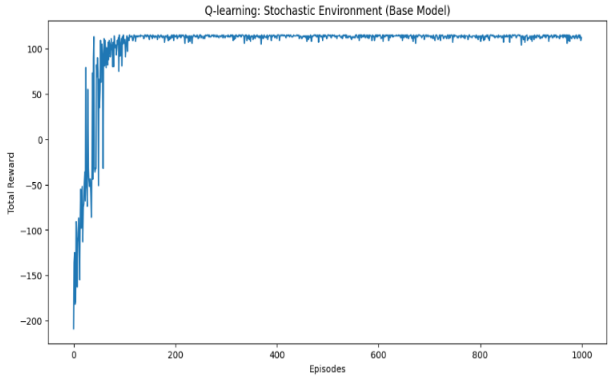


Fig 2.1 Q – Learning on Stochastic Warehouse

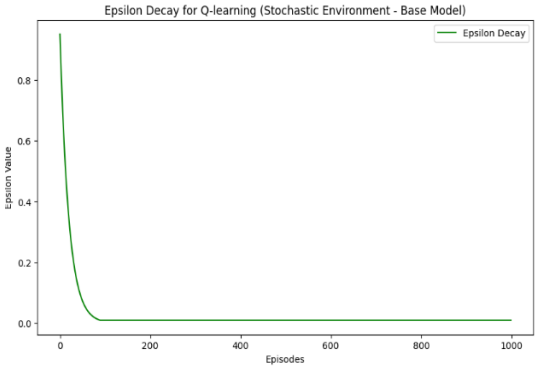


Fig 2.2 Epsilon Decay for Q-Learning

Observation:

Total Reward Plot (Stochastic Environment):

- Initially, the **rewards fluctuate quite a bit**, showing instability in the agent's learning. However, after several episodes, the **agent starts to converge to higher rewards** (approaching 100) indicating that **it has learned the optimal policy over time**.
- The fluctuations in the rewards are likely caused by the random events introduced in the stochastic environment, which makes the learning process noisier.

Epsilon Decay Plot (Stochastic Environment):

- The epsilon value decays sharply during the early episodes, **moving towards the minimum exploration** rate. This sharp decline indicates that the **model is quickly reducing exploration in favor of exploitation** as it learns, which is typical in Q-learning when epsilon decay is set to a high rate.
- Similar to the deterministic environment, the epsilon value stabilizes towards zero after around 100 episodes, which is when the agent mostly relies on the learned Q-values for decision making.

1(c) Applying SARSA on Deterministic Warehouse

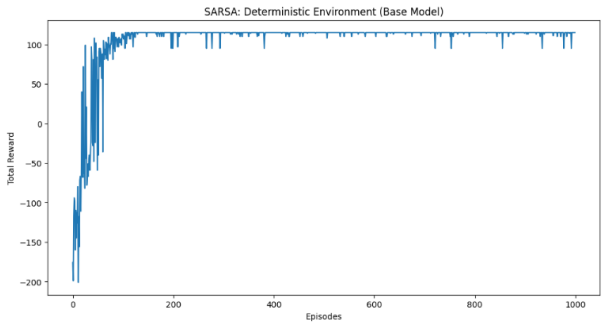


Fig 3.1 SARSA on Deterministic Warehouse

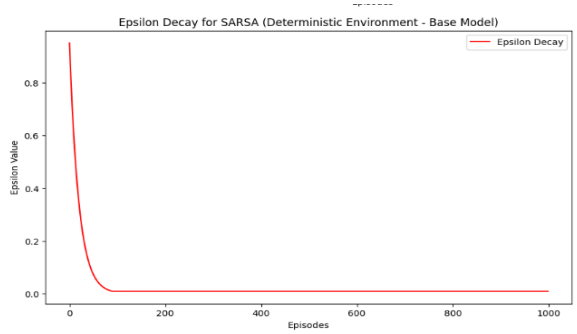


Fig 3.2 Epsilon decay on SARSA

Observation:

Total Reward Plot (Deterministic Environment) :

- The graph demonstrates that **SARSA** performs well in the deterministic environment, achieving a significant reward around **episode 200** and stabilizing thereafter. The agent successfully learned to navigate the environment and **accumulate positive rewards**, but there are fluctuations in earlier episodes, indicating the exploration phase where the algorithm is trying different actions and exploring states.

Epsilon Decay Plot (Deterministic Environment):

- The epsilon decay curve shows a sharp decline in the first few episodes and **gradually stabilizes**, approaching 0. This implies that the agent quickly reduced exploration in favor of exploiting the learned policy (greedy behavior) as training progresses. The epsilon value drops significantly around episode 100 and remains low, showing that most actions are now being chosen greedily rather than randomly.

1(d) Applying SARSA on Stochastic Warehouse

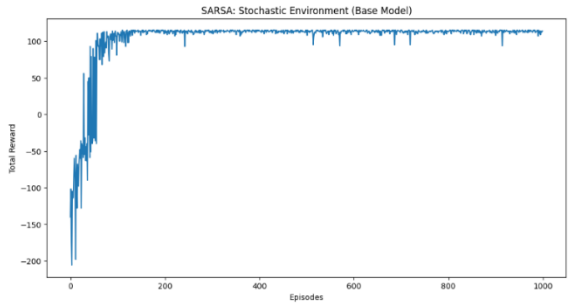


Fig 4.1 SARSA on Deterministic Warehouse

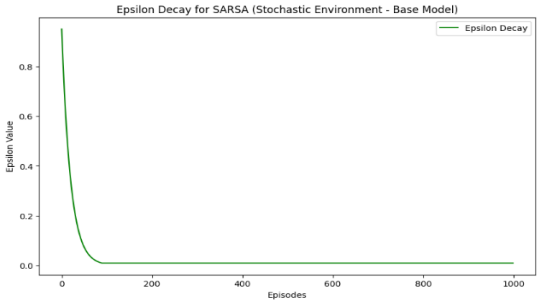


Fig 4.2 Epsilon Decay for SARSA

Total Reward Plot (Stochastic Environment):

- The total reward gradually increases as the episodes progress, indicating that the agent is learning to navigate the environment more effectively. Initially, the total reward is highly variable, likely due to random events (as it's a stochastic environment), but after several episodes, the reward stabilizes and approaches the maximum reward (100).
- There are some sharp drops in the reward curve, which could be due to the randomness in the environment or occasional exploration.

Epsilon Decay Plot (Stochastic Environment):

- The epsilon value starts high (near 1) **and decreases over time as expected**. This represents the transition from exploration to exploitation in the agent's learning process. After the initial exploration phase (first few hundred episodes), **epsilon decays towards the minimum value of 0**. This suggests that the agent eventually relies more on exploiting its learned policy rather than continuing to explore random actions.
- The smooth decay of epsilon aligns well with the agent's shift toward using its learned experience to maximize rewards in the environment.

1(e)

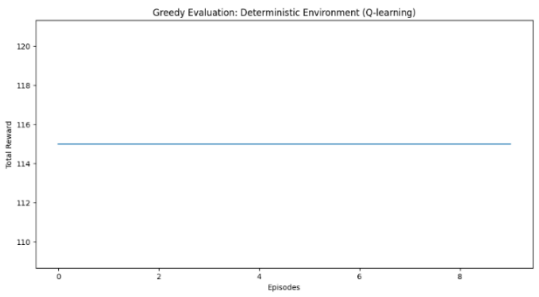


Fig 5.1 Greedy Evaluation for Deterministic Environment

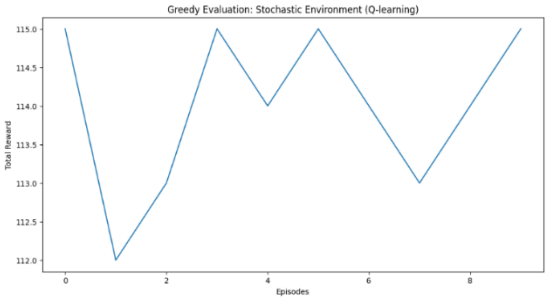


Fig 5.2 Greedy Evaluation for Stochastic Environment (Q-learning)

Greedy Evaluation for Deterministic Environment (Q-learning)

Observation (Fig 5.1):

- The **reward per episode** is **stable**, indicating that the agent has learned a good policy and is consistently taking the optimal actions. Since the rewards are constant and relatively high (around 114), it suggests that the agent has **converged to an effective greedy strategy** based on its prior experience.
- The lack of fluctuation in the total reward per episode indicates that the agent is exploiting the learned policy effectively, without making exploratory actions (as it's only choosing greedy actions).

Greedy Evaluation for Stochastic Environment (Q-learning)

Observation (Fig 5.2):

- Unlike the deterministic environment, the total reward per episode fluctuates between around 112.5 and 115.5, indicating some level of uncertainty or variance in the environment.
- The graph shows episodes with higher and lower rewards, with no smooth and consistent pattern. This is expected in a **stochastic environment**, where random elements can cause variations in the rewards from episode to episode.
- The random events (e.g., obstacles or changes in the environment) in the stochastic environment lead to **uncertainty** in the agent's performance, even when it's taking greedy actions based on the learned Q-values. This can be seen from the sharp fluctuations in the reward values across episodes.

1(f)

Fig 5.1 Rendering for Deterministic Env (Q-Learning)

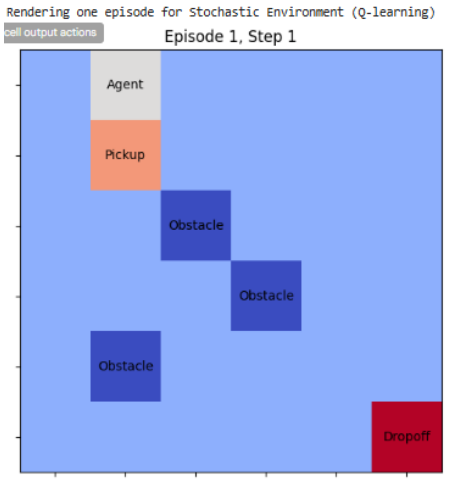


Fig 5.2 Rendering for Stochastic Env(Q-Learning)

Fig 5.3 Rendering for Deterministic Env (SARSA)

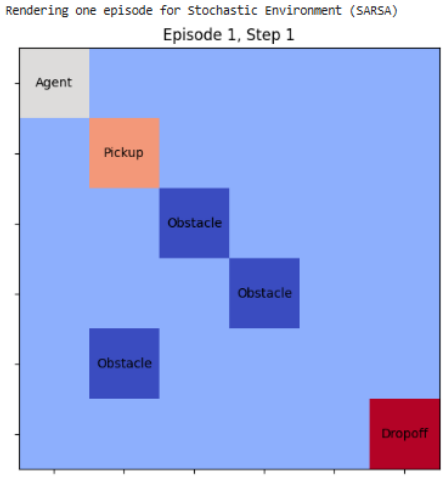


Fig 5.4 Rendering for Stochastic Env (SARSA)

Observation:

- In all cases, the agent successfully solves the environment by moving from the start to the goal, picking up and dropping off the item while avoiding obstacles. The renders show the agent's progress step-by-step, with clear visualizations of the environment and labels.

- The agent successfully completes its task using greedy actions from the learned policies, which indicates efficient exploration and exploitation of the environment.

2. Comparing the peroformance of SARSA and Q-Learning on Deterministic Environment:

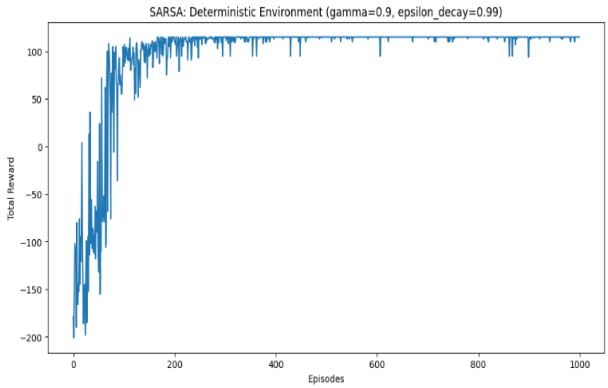


Fig 2.1. SARSA when (gamma = 0.9 and epsilon_decay = 0.99)

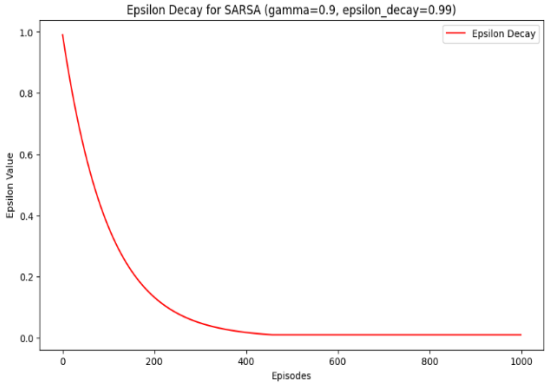


Fig 2.1. Epsilon Decay for SARSA

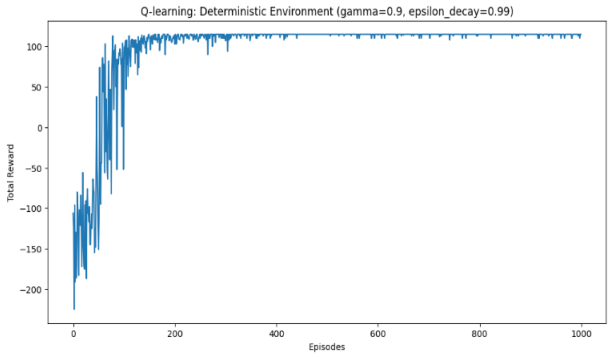


Fig 2.1. SARSA when (gamma = 0.9 and epsilon_decay = 0.99)

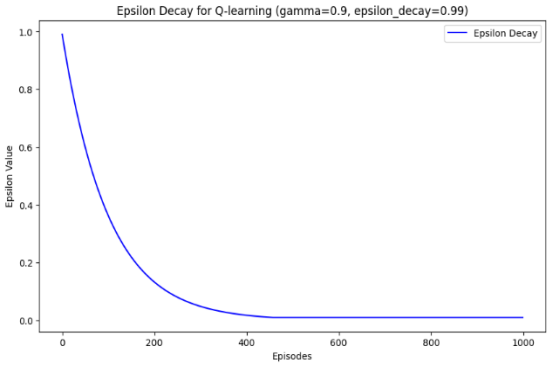
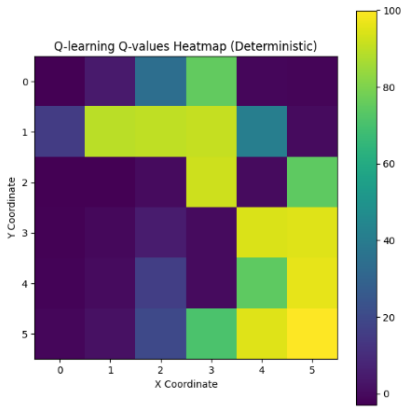
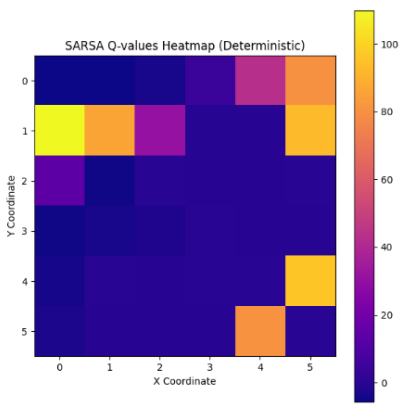


Fig 2.1. Epsilon Decay for SARSA

Comparison:

- **In SARSA** the reward dynamics in SARSA are more volatile, particularly in the initial phases of training, due to its on-policy nature. The agent explores and exploits based on its current policy, leading to slower convergence to the optimal policy. The reward improves over time, but it fluctuates more compared to Q-learning.
- **Q-learning**, being off-policy, performs better in terms of reward accumulation over time. It converges more quickly than SARSA because it learns the best actions independently of the agent's current policy, allowing for more efficient exploration. The reward dynamics are smoother and the rewards stabilize at higher levels faster than in SARSA.
- **Epsilon Decay** Both algorithms show similar epsilon decay trends. Initially, epsilon is high, encouraging exploration. Over time, epsilon decays to a low value, promoting exploitation of the learned policy as the agent becomes more confident in its actions.





3. Comparing the peroformance of SARSA and Q-Learning on Stochastic Environment

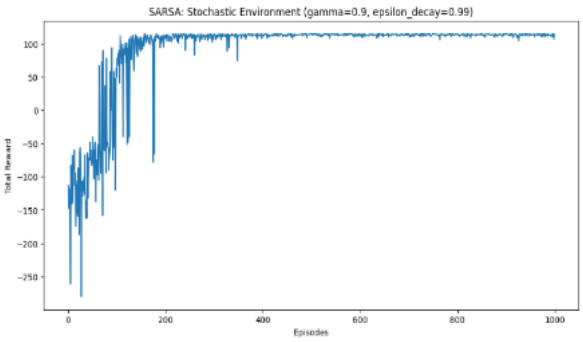


Fig 2.1. SARSA when (gamma = 0.9 and epsilon_decay = 0.99)

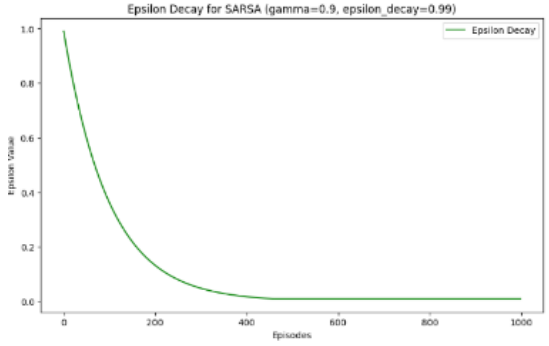


Fig 2.1. Epsilon Decay for SARSA

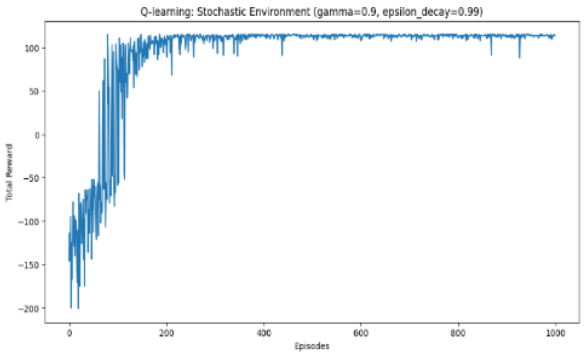


Fig 2.1. SARSA when (gamma = 0.9 and epsilon_decay = 0.99)

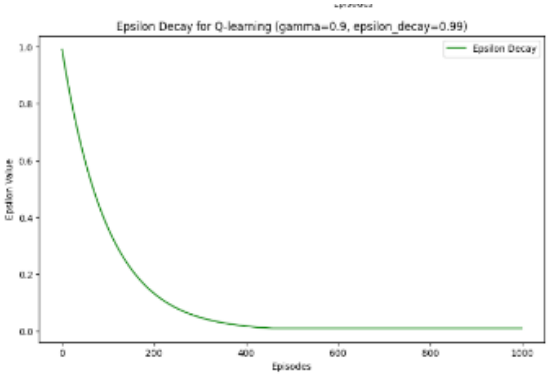
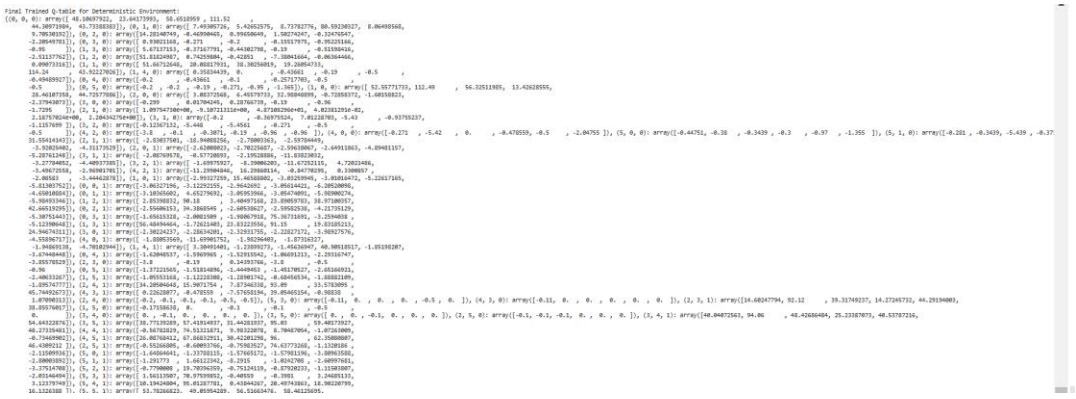


Fig 2.1. Epsilon Decay for SARSA

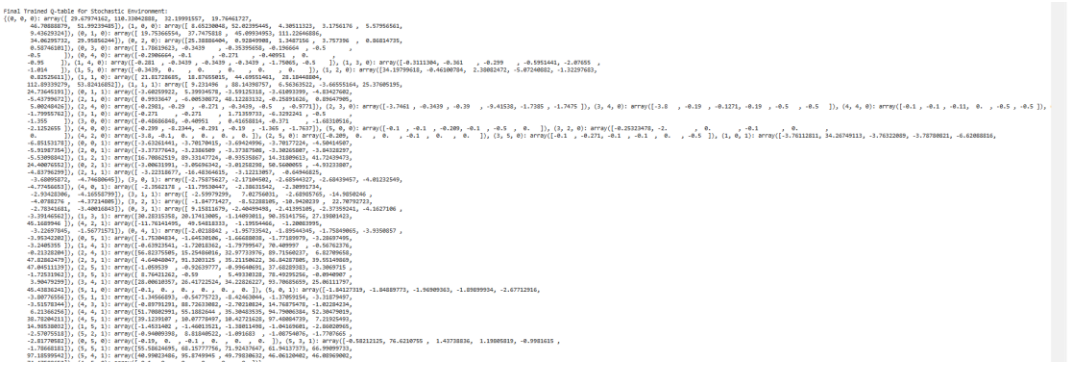
Comparison:

- Q-learning appears to converge more steadily, likely because it learns the optimal policy through off-policy learning (evaluating future rewards). In contrast, SARSA exhibits more erratic behavior since it learns from the actions it actually takes in each episode, adjusting its actions based on the immediate environment.Both algorithms show similar epsilon decay patterns, with epsilon decreasing rapidly initially and then stabilizing. However, SARSA's learning is more sensitive to the exploration-exploitation trade-off because it uses the epsilon value in its own action decision-making process, while Q-learning uses its epsilon value in action selection during updates, which could lead to a slightly faster convergence.
- **Q-learning** shows better stability and convergence in the stochastic environment, while **SARSA** may take longer to stabilize due to its on-policy nature, which causes more exploration and fluctuation in reward accumulation.

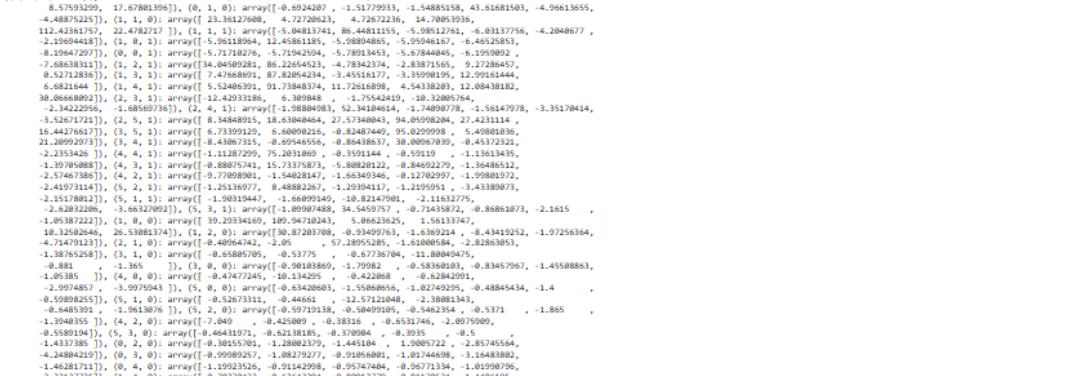
4. Final Trained Q-table for Deterministic Environment:



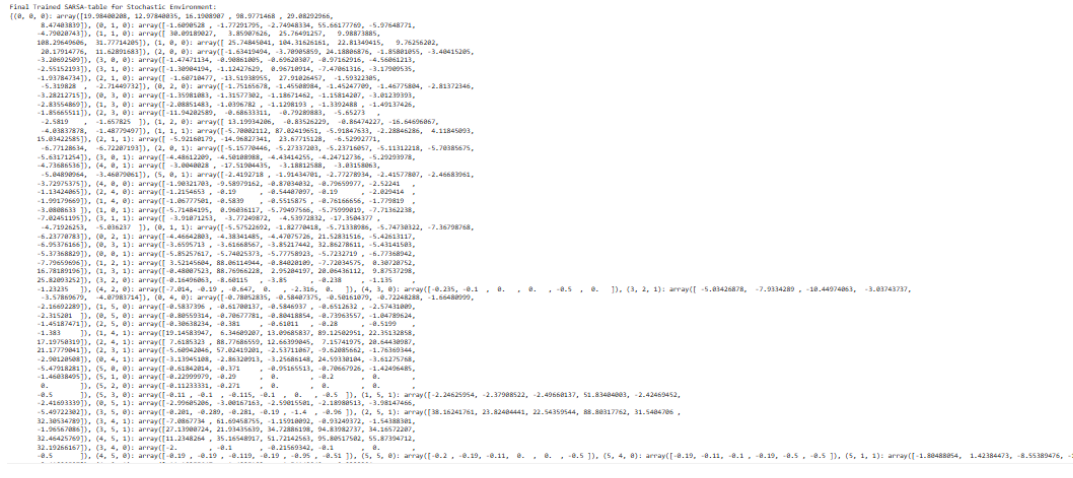
Final Trained Q-table for Stochastic Environment:



Final Trained SARSA-table for Deterministic Environmt:



Final Trained SARSA-table for Stochastic Environment:



Part 3: Solve Stock Trading Environment

Fig 1.1 Total Rewards per Episode

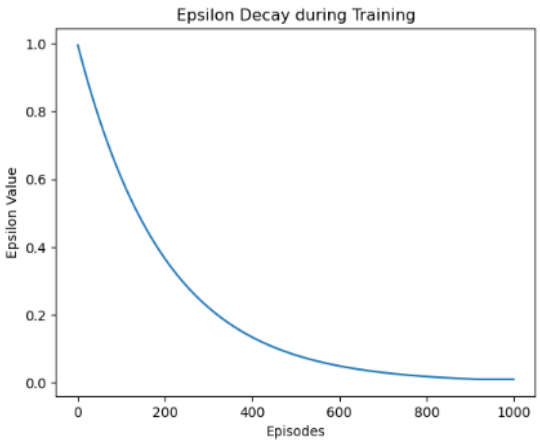


Fig 1.2 Epsilon Decay during training

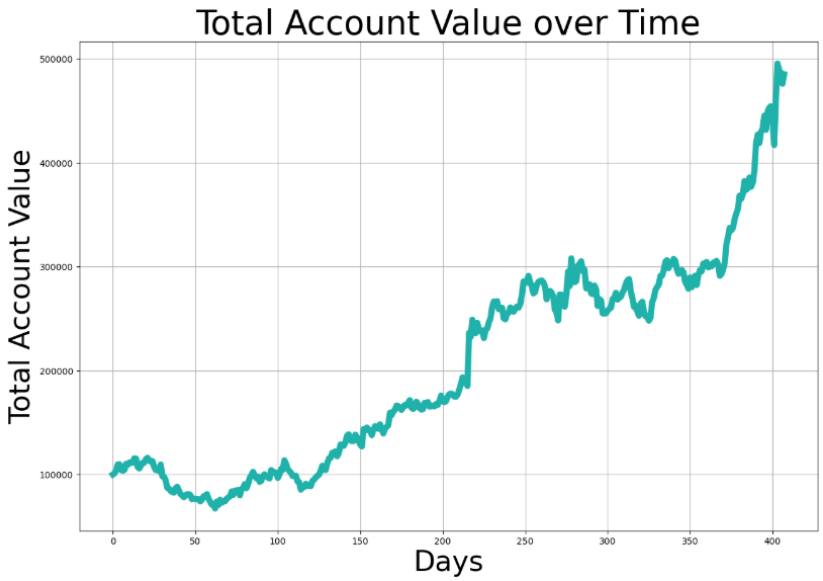
Observation:

Total Rewards per Episode():

- The total rewards increase steadily as the episodes progress, indicating that the agent is learning and improving its performance over time.
- There are some fluctuations within the reward dynamics, especially in the earlier episodes, which is expected as the agent is exploring and adjusting its strategy.

Epsilon Decay:

- The epsilon value decays smoothly from 1.0 to a small value near 0. This gradual decay suggests that the agent is initially exploring the environment and gradually shifting towards exploiting the learned policy as it gains more experience.
- The decay curve is typical for reinforcement learning where the agent starts with exploration (high epsilon) and later relies on exploitation (low epsilon).



Observation:

- In the early stages, the account value experiences fluctuations, which are common as the agent starts exploiting its policy. After some time, the account value starts to grow steadily, indicating that the agent is making increasingly better decisions.
- Towards the end, there is a sharp increase in the total account value, which suggests that the agent has learned an effective strategy for maximizing profits over time.
- This result shows that the agent, after training, is capable of making decisions that lead to substantial profits, demonstrating the effectiveness of Q-learning in solving the stock trading problem.

References

- RL Handbook
- NIPS Styles (docx, tex)
- GYM environments
- Lecture slides
- Richard S. Sutton and Andrew G. Barto, "Reinforcement learning: An introduction" (pdf)